

EPIC 05 — Task 05: Resilient Microservices

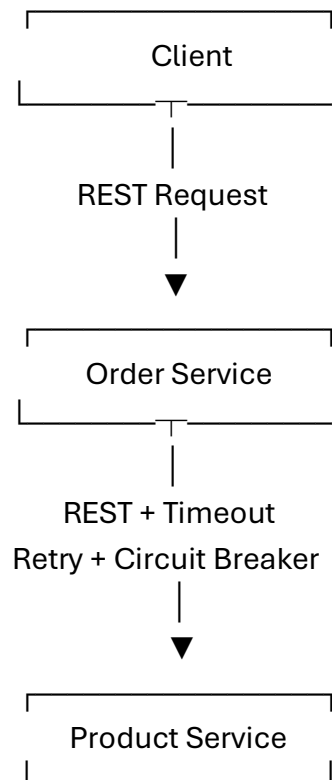
Workflow

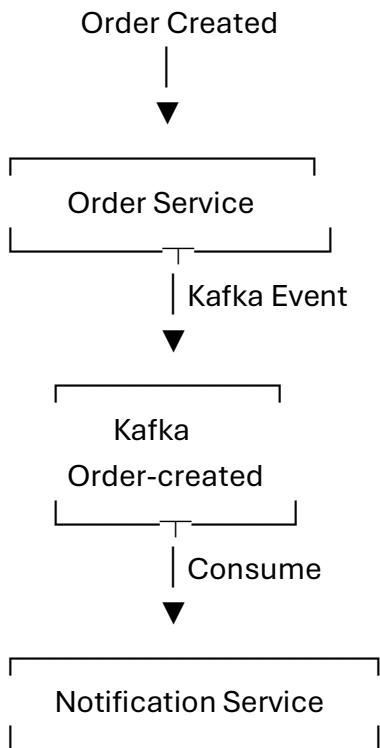
The task requires combining:

- REST
- Retry
- Timeout
- Circuit Breaker
- Kafka
- Fallback
- Failure testing
- Architecture documentation

We can build this using the **Order Service**, **Product Service**, **Notification Service**, and **Kafka** you created in Tasks 01–04.

Step 1 — Final Architecture





Step 2 — Combine REST + Retry + Timeout + Circuit Breaker

Your ProductClient.java should contain both Retry and Circuit Breaker.

```
package com.example.orderservice.client;
```

```
import org.springframework.stereotype.Component;
```

```
import org.springframework.web.client.RestClient;
```

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
```

```
import io.github.resilience4j.retry.annotation.Retry;
```

```
@Component
```

```
public class ProductClient {
```

```
    private final RestClient restClient;
```

```
    public ProductClient(RestClient restClient) {
```

```

        this.restClient = restClient;
    }

    @Retry(name = "productService")
    @CircuitBreaker(
        name = "productService",
        fallbackMethod = "getProductFallback"
    )
    public ProductResponse getProductById(Long productId) {

        return restClient.get()
            .uri("/products/{id}", productId)
            .retrieve()
            .body(ProductResponse.class);
    }

    public ProductResponse getProductFallback(
        Long productId,
        Exception exception) {

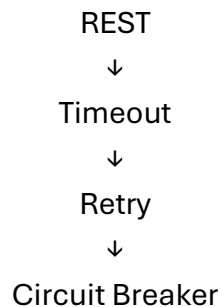
        ProductResponse response = new ProductResponse();

        response.setId(productId);
        response.setName("Product Service Unavailable");
        response.setPrice(0.0);

        return response;
    }
}

```

So, the request now has this protection:



↓
Product Service

Step 3 — Configure Timeout

In:

src/main/java/com/example/orderservice/config/RestClientConfig.java

Use:

```
package com.example.orderservice.config;

import java.time.Duration;

import org.apache.hc.client5.http.config.RequestConfig;
import org.apache.hc.client5.http.impl.classic.CloseableHttpClient;
import org.apache.hc.client5.http.impl.classic.HttpClients;
import org.apache.hc.client5.http.impl.io.PoolingHttpClientConnectionManagerBuilder;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.HttpComponentsClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

@Configuration
public class RestClientConfig {

    @Bean
    public RestClient restClient(RestClient.Builder builder) {

        RequestConfig requestConfig = RequestConfig.custom()
            .setConnectTimeout(3000)
            .setResponseTimeout(5000)
            .build();

        CloseableHttpClient httpClient = HttpClients.custom()
            .setDefaultRequestConfig(requestConfig)
            .setConnectionManager(
```

```

        PoolingHttpClientConnectionManagerBuilder.create()
            .build()
    )
    .build();

    HttpClientComponentsClientHttpRequestFactory factory =
        new HttpClientComponentsClientHttpRequestFactory(httpClient);

    return builder
        .requestFactory(factory)
        .baseUrl("http://localhost:8082")
        .build();
    }
}

```

This gives:

Connection timeout = 3 seconds
 Read/response timeout = 5 seconds

Step 4 — Configure Retry

In:

src/main/resources/application.properties

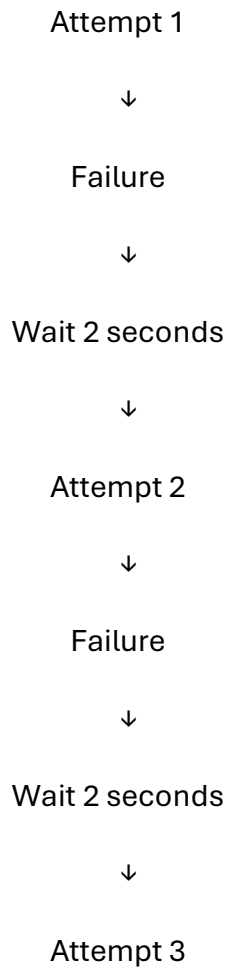
add:

resilience4j.retry.instances.productService.max-attempts=3

resilience4j.retry.instances.productService.wait-duration=2s

resilience4j.retry.instances.productService.retry-
 exceptions=org.springframework.web.client.ResourceAccessException,org.springframework
 web.client.HttpServerErrorException

The system will try a failed request up to **3 attempts**.



Step 5 — Configure Circuit Breaker

Add:

```
resilience4j.circuitbreaker.instances.productService.sliding-window-type=COUNT_BASED  
resilience4j.circuitbreaker.instances.productService.sliding-window-size=5
```

```
resilience4j.circuitbreaker.instances.productService.minimum-number-of-calls=5
```

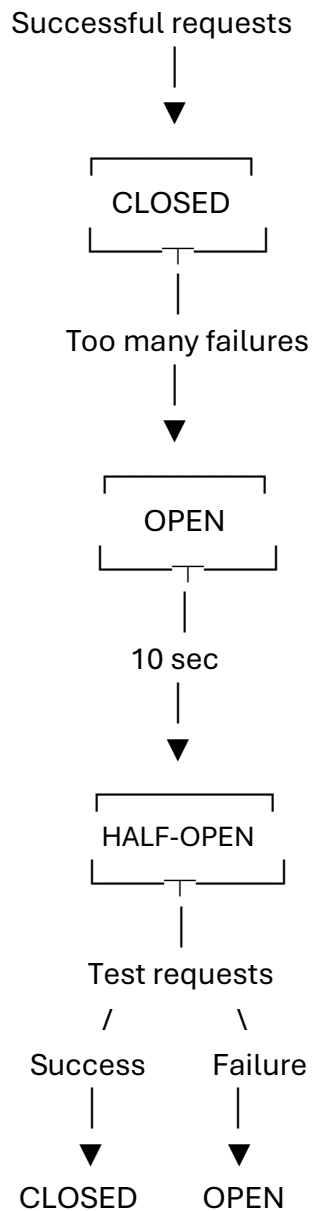
```
resilience4j.circuitbreaker.instances.productService.failure-rate-threshold=50
```

resilience4j.circuitbreaker.instances.productService.wait-duration-in-open-state=10s

resilience4j.circuitbreaker.instances.productService.permitted-number-of-calls-in-half-open-state=2

resilience4j.circuitbreaker.instances.productService.automatic-transition-from-open-to-half-open-enabled=true

The circuit works like this:



Step 6 — Configure Kafka

Your Order Service should publish an event when an order is successfully created.

application.properties:

```
spring.kafka.bootstrap-servers=localhost:9092
```

```
spring.kafka.producer.key-
```

```
serializer=org.apache.kafka.common.serialization.StringSerializer
```

```
spring.kafka.producer.value-
```

```
serializer=org.springframework.kafka.support.serializer.JsonSerializer
```

Step 7 — Publish Order Event

Your producer:

```
@Service
```

```
public class OrderEventProducer {
```

```
    private static final String TOPIC = "order-created";
```

```
    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;
```

```
    public OrderEventProducer(  
        KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {
```

```
        this.kafkaTemplate = kafkaTemplate;  
    }
```

```
    public void publishOrderCreated(OrderCreatedEvent event) {
```

```
        kafkaTemplate.send(
```

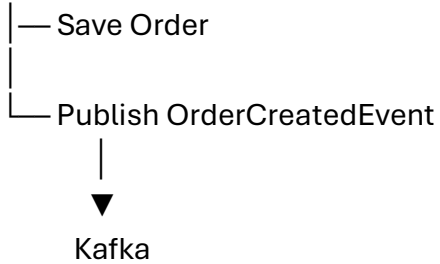
```

        TOPIC,
        event.getOrderId().toString(),
        event
    );
}
}

```

When an order is created:

Order Service



Step 8 — Notification Service Consumes Kafka Event

In Notification Service:

```

@Service
public class OrderEventConsumer {

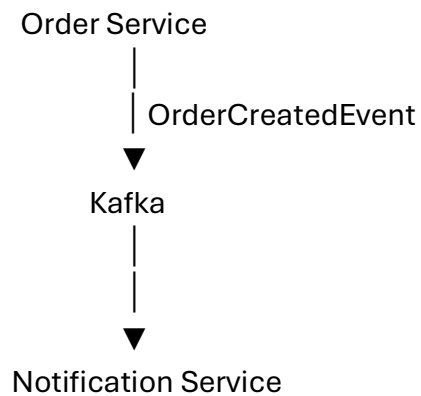
    @KafkaListener(
        topics = "order-created",
        groupId = "notification-service"
    )
    public void consume(OrderCreatedEvent event) {

        System.out.println(
            "Order received: " + event.getOrderId()
        );

        System.out.println(
            "Sending notification for order: "
            + event.getOrderId()
        );
    }
}

```

The final event flow is:



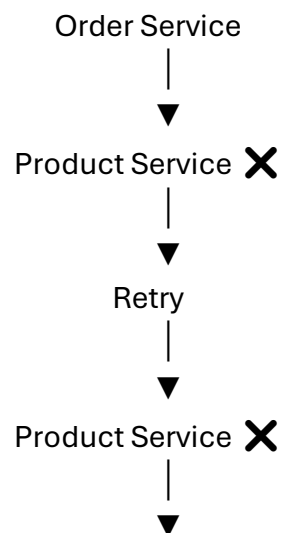
Step 9 — Test Failure Scenario 1: Product Service Down

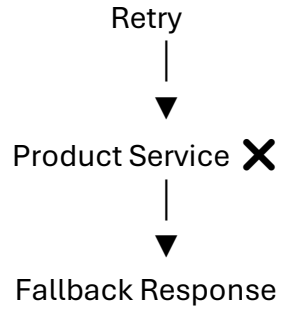
Stop the Product Service.

Then call:

GET <http://localhost:8081/orders/product/1>

Expected behavior:





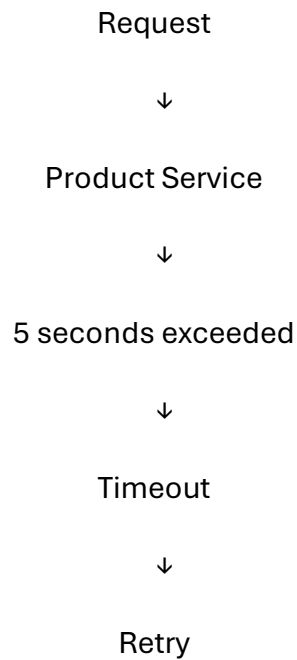
Expected response:

```
{
  "id": 1,
  "name": "Product Service Unavailable",
  "price": 0.0
}
```

Step 10 — Test Failure Scenario 2: Timeout

Make Product Service take longer than the configured **5-second response timeout**.

Expected:



After the retry limit is reached, the fallback response is returned.

Step 11 — Test Failure Scenario 3: Circuit Breaker

Stop Product Service and send several requests.

After the configured failure threshold is reached:

CLOSED



Failures



OPEN

Now requests are rejected by the circuit breaker without repeatedly calling Product Service.

After **10 seconds**:

OPEN



HALF-OPEN

Two test calls are allowed.

If Product Service has recovered:

HALF-OPEN



SUCCESS



CLOSED

Step 12 — Test Kafka Failure

Stop the Notification Service.

Create an order.

The Order Service can still publish:



When Notification Service starts again, it can consume the available Kafka event according to the consumer configuration.

This demonstrates the advantage of **event-driven communication**: Order Service does not need to synchronously wait for Notification Service.

Step 13 — Final Architecture Documentation

Resilient Microservices Workflow

The system uses REST-based synchronous communication between Order Service and Product Service. RestClient is used for HTTP communication with connection and response timeouts. Resilience4j Retry provides controlled retries for transient failures, while Circuit Breaker prevents repeated calls to an unhealthy Product Service. A fallback response is returned when the Product Service remains unavailable. For asynchronous communication, Order Service publishes an OrderCreated event to Kafka, and Notification Service consumes the event and processes the notification.

Requirement	Implementation
REST	RestClient
Timeout	3-second connection / 5-second response
Retry	Maximum 3 attempts
Circuit Breaker	Closed / Open / Half-Open
Fallback	getProductFallback()

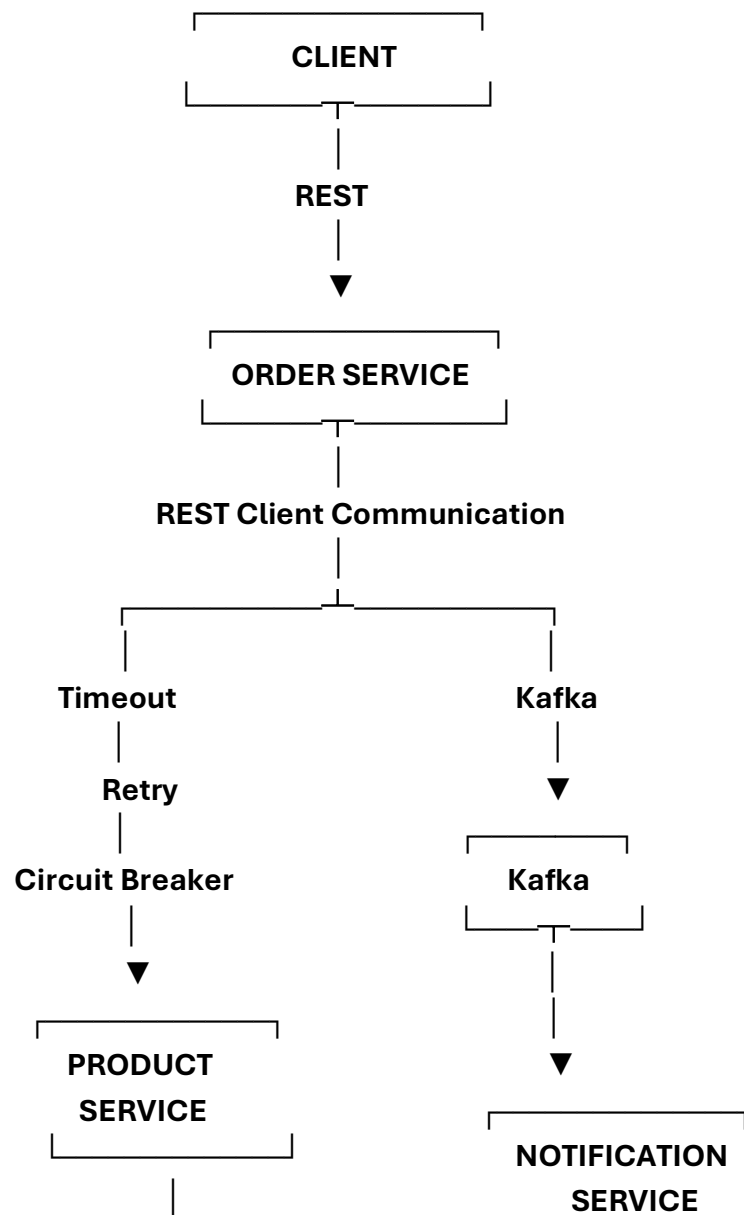
Kafka
Event publishing
Event consumption

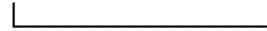
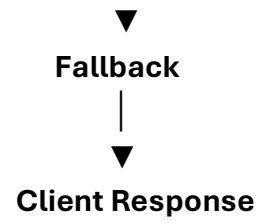
Failure testing

Architecture documentation

order-created topic
Order Service
Notification Service
Product failure, timeout, circuit open,
Kafka scenario
Completed

Complete flow:





Final integration task for

EPIC 05: REST + Timeout + Retry + Circuit Breaker + Kafka + Fallback.